

MySQL – Practical Examination

Structured Query Language (SQL)

Database: **ShopSphere** (Retail) | Duration: **1 Hour** | Max Marks: **50**

Name: _____

Roll No.: _____

Date: _____

Signature: _____

General Instructions

1. This test is based on the **ShopSphere** sample database. The schema and sample rows are given on the reference page (last page). Study it before answering.
2. Tables: categories, suppliers, customers, employees, products, orders, order_items, payments.
3. Write **standard MySQL** syntax. Marks are awarded for correct logic even if minor syntax slips occur.
4. Attempt **all** sections. Section A (10), B (10), C (15), D (15) — total 50 marks.
5. Manage your time: A—10 min, B—10 min, C—20 min, D—20 min.

SECTION A — Objective / Theory (10 × 1 = 10 Marks)

Answer briefly. For MCQs, write the correct option letter.

- A1) Which SQL sub-language does the CREATE TABLE statement belong to? (a) DML (b) DDL (c) DCL (d) TCL
- A2) What is the difference between WHERE and HAVING clauses?
- A3) Name any two aggregate functions in SQL.
- A4) Which clause is used to remove duplicate rows from a result set?
- A5) State the difference between DELETE, TRUNCATE and DROP.
- A6) What is a **PRIMARY KEY**? Can it contain NULL values?
- A7) Predict the output type: SELECT COUNT(*) FROM customers; returns a single _____ value.
- A8) Which JOIN returns all rows from the left table and matching rows from the right table?
- A9) What is the purpose of the LIMIT clause in MySQL?
- A10) True/False: A FOREIGN KEY can reference the PRIMARY KEY of the same table.

SECTION B — DDL & DML (5 × 2 = 10 Marks)

Write the SQL statement for each.

- B1) Create a table reviews with columns: review_id (primary key, auto-increment), product_id (integer, foreign key referencing products), rating (integer, must be between 1 and 5), and comment (up to 255 chars).
- B2) Insert a new customer: Meera Nanda, email meera.n@mail.com, city Surat, state Gujarat, who signed up today.
- B3) Increase the price of every product in the **Electronics** category by 5%.
- B4) Add a new column phone VARCHAR(15) to the customers table.
- B5) Delete all orders whose status is 'Cancelled'.

SECTION C — Queries: SELECT, Filtering, Joins (5 × 3 = 15 Marks)

Write a single query for each requirement.

- C1) List the `first_name`, `last_name` and `city` of all customers from **Maharashtra**, sorted alphabetically by first name.
- C2) Show each product name along with its category name (use a JOIN between products and categories).
- C3) Find all products whose price is between 1000 and 5000, and that are currently in stock (`stock_qty > 0`).
- C4) Display every order with the customer's full name and the name of the employee who handled it (join orders, customers and employees).
- C5) List all customers who have **never** placed an order (use a LEFT JOIN or NOT IN).

SECTION D — Aggregation, Subqueries, Views (5 × 3 = 15 Marks)

These test grouping, subqueries and advanced features.

- D1) For each order status, show the number of orders (use GROUP BY).
- D2) Show the total revenue per order — i.e. for each `order_id`, the sum of `quantity × unit_price` from `order_items`. Show only orders whose total exceeds 5000 (use HAVING).
- D3) Find the product(s) with the highest price using a subquery.
- D4) Show each category name and the average price of products in it; order from highest average to lowest.
- D5) Create a **VIEW** named `vw_customer_spend` that lists each customer's full name and their total amount paid (join customers → orders → payments).

— End of Question Paper —

Reference: ShopSphere Schema & Sample Data

Table Structures

```
categories(category_id PK, category_name, description)
suppliers(supplier_id PK, supplier_name, city, country, rating)
customers(customer_id PK, first_name, last_name, email, city, state, signup_date)
employees(employee_id PK, first_name, last_name, department, salary, hire_date,
manager_id FK->employees)
products(product_id PK, product_name, category_id FK, supplier_id FK, price,
stock_qty)
orders(order_id PK, customer_id FK, employee_id FK, order_date, status)
order_items(order_id FK, product_id FK, quantity, unit_price) [PK:
order_id+product_id]
payments(payment_id PK, order_id FK, amount, method, payment_date)
```

categories

id	category_name	description
1	Electronics	Phones, laptops, gadgets
2	Home & Kitchen	Cookware, appliances
3	Books	Fiction, non-fiction, academic
4	Clothing	Men, women, kids apparel
5	Sports	Fitness, outdoor gear
6	Toys	Toys and games

customers

id	first	last	email	city	state	signup
1	Aarav	Sharma	aarav.sharma@mail.com	Mumbai	Maharashtra	2023-01-15
2	Diya	Patel	diya.patel@mail.com	Ahmedabad	Gujarat	2023-02-20
3	Kabir	Singh	kabir.singh@mail.com	Delhi	Delhi	2023-03-05
4	Anaya	Reddy	anaya.reddy@mail.com	Hyderabad	Telangana	2023-03-25
5	Vivaan	Nair	vivaan.nair@mail.com	Kochi	Kerala	2023-05-10
6	Ishita	Gupta	ishita.gupta@mail.com	Delhi	Delhi	2023-06-18
7	Arjun	Mehta	arjun.mehta@mail.com	Mumbai	Maharashtra	2023-07-22
8	Saanvi	Iyer	(NULL)	Chennai	Tamil Nadu	2023-08-30
9	Rohan	Verma	rohan.verma@mail.com	Pune	Maharashtra	2023-09-12
10	Myra	Joshi	myra.joshi@mail.com	Jaipur	Rajasthan	2023-11-01

products

id	product_name	cat	sup	price	stock
1	Smartphone X200	1	1	24999.00	40
2	Wireless Earbuds Pro	1	1	4999.00	120
3	14" Laptop UltraBook	1	1	58999.00	15
4	Non-Stick Cookware Set	2	2	2499.00	60
5	Electric Kettle 1.5L	2	2	1299.00	80
6	Stainless Water Bottle	2	2	499.00	200
7	SQL Fundamentals Book	3	3	650.00	35
8	Mystery Novel Boxset	3	3	1200.00	25
9	Men's Cotton T-Shirt	4	4	799.00	150
10	Women's Running Shoes	4	4	2999.00	45
11	Yoga Mat Premium	5	5	999.00	70
12	Adjustable Dumbbells	5	5	4499.00	20
13	Building Blocks 500pc	6	6	1499.00	55
14	Remote Control Car	6	6	1899.00	0

employees

id	first	last	department	salary	hire_date	mgr
1	Neha	Kapoor	Management	120000	2021-01-10	(NULL)
2	Raj	Malhotra	Sales	65000	2021-03-15	1
3	Priya	Desai	Sales	58000	2021-06-01	2
4	Sameer	Khan	Support	52000	2022-02-20	1
5	Tara	Bose	Support	47000	2022-05-11	4
6	Karan	Sethi	Warehouse	43000	2022-08-19	1
7	Aisha	Rao	Sales	61000	2023-01-05	2

orders

id	cust	emp	order_date	status
1	1	2	2024-01-05	Delivered
2	2	3	2024-01-12	Delivered
3	1	2	2024-02-03	Shipped
4	3	7	2024-02-15	Delivered
5	4	3	2024-03-01	Cancelled
6	5	2	2024-03-18	Delivered
7	6	7	2024-04-02	Pending
8	7	3	2024-04-20	Delivered
9	1	2	2024-05-06	Delivered
10	9	7	2024-05-22	Shipped
11	3	3	2024-06-10	Delivered
12	10	2	2024-06-28	Pending

order_items

order	product	qty	unit_price
1	1	1	24999.00
1	2	2	4999.00
2	7	3	650.00
2	4	1	2499.00
3	3	1	58999.00
4	9	4	799.00
4	10	1	2999.00
5	5	2	1299.00
6	6	5	499.00
6	11	1	999.00
7	13	2	1499.00
8	2	1	4999.00
8	8	1	1200.00
9	12	1	4499.00
9	11	2	999.00
10	1	1	24999.00
11	7	2	650.00
11	6	3	499.00
12	10	1	2999.00

payments

id	order	amount	method	payment_date
1	1	34997.00	Card	2024-01-05
2	2	4449.00	UPI	2024-01-12
3	3	58999.00	NetBanking	2024-02-03
4	4	6195.00	COD	2024-02-16
5	6	2598.00	UPI	2024-03-18
6	8	6199.00	Card	2024-04-20
7	9	6497.00	UPI	2024-05-06
8	10	24999.00	NetBanking	2024-05-22
9	11	2797.00	Card	2024-06-10

suppliers: 6 rows (TechWorld, HomeLine, PageTurner, UrbanStyle, FitGear, PlayZone). PlayZone has rating = NULL.

ANSWER KEY & MARKING SCHEME

(For examiner / self-assessment use)

Section A

A1

(b) DDL

A2

WHERE filters individual rows BEFORE grouping; HAVING filters groups AFTER GROUP BY (works on aggregates).

A3

Any two of: COUNT(), SUM(), AVG(), MIN(), MAX().

A4

DISTINCT.

A5

DELETE removes rows (DML, can use WHERE, logged, rollback-able); TRUNCATE removes ALL rows fast (DDL, resets AUTO_INCREMENT); DROP removes the whole table structure.

A6

A PRIMARY KEY uniquely identifies each row; it must be UNIQUE and NOT NULL (cannot contain NULLs).

A7

a single scalar / integer value.

A8

LEFT JOIN (LEFT OUTER JOIN).

A9

LIMIT restricts the number of rows returned by the query.

A10

True (a self-referencing / recursive foreign key, e.g. employees.manager_id).

Section B

B1

```
CREATE TABLE reviews (  
    review_id INT PRIMARY KEY AUTO_INCREMENT,  
    product_id INT,  
    rating INT CHECK (rating BETWEEN 1 AND 5),  
    comment VARCHAR(255),  
    FOREIGN KEY (product_id) REFERENCES products(product_id)  
);
```

B2

```
INSERT INTO customers (first_name, last_name, email, city, state, signup_date)  
VALUES ('Meera', 'Nanda', 'meera.n@mail.com', 'Surat', 'Gujarat', CURDATE());
```

B3

```
UPDATE products  
SET price = price * 1.05  
WHERE category_id = (SELECT category_id FROM categories  
                     WHERE category_name = 'Electronics');
```

B4

```
ALTER TABLE customers ADD COLUMN phone VARCHAR(15);
```

B5

```
DELETE FROM orders WHERE status = 'Cancelled';
```

Section C

C1

```
SELECT first_name, last_name, city
FROM customers
WHERE state = 'Maharashtra'
ORDER BY first_name;
```

C2

```
SELECT p.product_name, c.category_name
FROM products p
JOIN categories c ON p.category_id = c.category_id;
```

C3

```
SELECT product_name, price, stock_qty
FROM products
WHERE price BETWEEN 1000 AND 5000
      AND stock_qty > 0;
```

C4

```
SELECT o.order_id,
       CONCAT(c.first_name, ' ', c.last_name) AS customer,
       CONCAT(e.first_name, ' ', e.last_name) AS employee
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
LEFT JOIN employees e ON o.employee_id = e.employee_id;
```

C5

```
SELECT c.customer_id, c.first_name, c.last_name
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
-- (Customers 2, 4, 6, 8 have orders; those with none appear here.)
```

Section D

D1

```
SELECT status, COUNT(*) AS num_orders
FROM orders
GROUP BY status;
```

D2

```
SELECT order_id, SUM(quantity * unit_price) AS order_total
FROM order_items
GROUP BY order_id
HAVING SUM(quantity * unit_price) > 5000;
```

D3

```
SELECT product_name, price
FROM products
WHERE price = (SELECT MAX(price) FROM products);
```

D4

```
SELECT c.category_name, AVG(p.price) AS avg_price
FROM categories c
JOIN products p ON c.category_id = p.category_id
GROUP BY c.category_name
ORDER BY avg_price DESC;
```

D5

```
CREATE VIEW vw_customer_spend AS
SELECT c.customer_id,
       CONCAT(c.first_name, ' ', c.last_name) AS customer,
       SUM(pay.amount) AS total_paid
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN payments pay ON o.order_id = pay.order_id
GROUP BY c.customer_id, customer;
```